1.1 Tổng quan về lỗ hổng Command Injection

Command Injection là một trong những lỗ hổng bảo mật nghiêm trọng có độ phổ biến cao trên các ứng dụng web tương tác trực tiếp với hệ điều hành máy chủ. Lỗ hổng này phát sinh khi ứng dụng tiếp nhận dữ liệu đầu vào từ người dùng (thông qua các tham số GET, POST, HTTP headers hoặc cookies) nhưng không được kiểm tra, lọc ký tự hoặc chuẩn hóa đầy đủ trước khi truyền trực tiếp vào các hàm thực thi hệ thống của ngôn ngữ lập trình như system(), exec() hay shell_exec(). Khi kẻ tấn công chèn các ký tự điều khiển shell đặc biệt như dấu chấm phẩy ";", toán tử pipe "|", dấu và "&", backticks "`", hay cú pháp nội suy "$()", luồng thực thi ban đầu của ứng dụng bị bẻ gãy. Hệ điều hành sẽ hiểu nhầm dữ liệu độc hại do người dùng cung cấp chính là một phần của câu lệnh hệ thống và tiến hành thực thi chúng với quyền hạn của tiến trình web server, thường là tài khoản www-data trên hệ thống Linux.

Việc khai thác thành công lỗ hổng Command Injection cho phép kẻ tấn công thực hiện các hành vi phá hoại trực tiếp vào hạ tầng, bao gồm đọc và đánh cắp các tệp tin nhạy cảm như /etc/passwd, thực thi mã độc tùy ý, chiếm quyền kiểm soát tiến trình hoặc mở rộng phạm vi tấn công sang các dịch vụ nội bộ khác trong mạng. Do đó, việc triển khai các giải pháp bảo vệ ở tầng ứng dụng, đặc biệt là hệ thống tường lửa ứng dụng web Web Application Firewall như ModSecurity kết hợp với bộ luật chuẩn quốc tế OWASP Core Rule Set, đóng vai trò là lớp khiên phòng thủ tuyến đầu vô cùng quan trọng để ngăn chặn từ gốc các nguy cơ tấn công này.

1.2 Mục tiêu

Bài thực nghiệm được triển khai nhằm mục đích xây dựng, cấu hình và đánh giá toàn diện năng lực phòng thủ của hệ thống Web Application Firewall sử dụng Nginx và ModSecurity đối với lỗ hổng Command Injection trên ứng dụng mẫu DVWA. Quá trình kiểm thử được thiết kế trải qua 5 giai đoạn vận hành liên tiếp trên cùng một tệp cấu hình trung tâm nhằm khảo sát chi tiết sự khác biệt giữa các cơ chế bảo vệ:

Giai đoạn 1 (Off): Khảo sát hành vi của ứng dụng khi hoàn toàn không có WAF bảo vệ, qua đó xác nhận sự tồn tại và mức độ dễ bị tổn thương của lỗ hổng Command Injection.

Giai đoạn 2 (DetectionOnly - Custom Rule): Đánh giá chế độ giám sát thầm lặng, nơi ModSecurity sử dụng luật tùy chỉnh để nhận diện và ghi nhận sự kiện độc hại vào hệ thống tệp log nhưng không can thiệp vào luồng request của người dùng.

Giai đoạn 3 (Blocking - Custom Rule): Kiểm chứng khả năng phòng thủ chủ động của luật tùy chỉnh khi chuyển engine sang trạng thái kích hoạt , tiến hành chặn đứng request độc hại và trả về mã lỗi HTTP 403 Forbidden.

Giai đoạn 4 (Blocking - OWASP CRS): Kiểm chứng khả năng bảo vệ tổng quát và mạnh mẽ của OWASP CRS khi vô hiệu hóa hoàn toàn luật tùy chỉnh, đánh giá cơ chế chấm điểm bất thường để từ chối truy cập trái phép.

1.3 Chuẩn bị môi trường thực nghiệm

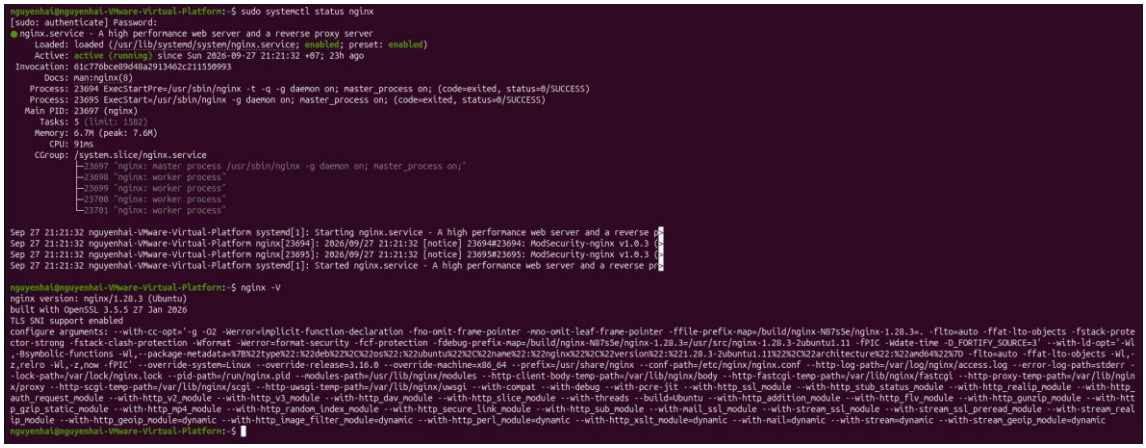
Môi trường triển khai: Hệ điều hành Ubuntu Linux, Nginx lắng nghe tại cổng 80, và ứng dụng DVWA chạy bên trong môi trường container hóa Docker tại cổng 8080.

Cấu hình trung tâm: Sử dụng duy nhất một tệp cấu hình được thiết kế với tính linh hoạt cao. Quá trình chuyển đổi giữa các trạng thái giám sát và chặn thực tế , cũng như việc bật/tắt các lớp quy tắc , được thực hiện nhanh chóng thông qua việc điều chỉnh thông số engine.

1.4 Các bước thực hiện

1.4.1 Thiết lập ban đầu và xác thực môi trường ứng dụng

Trước khi tiến hành kiểm thử các lớp phòng thủ của WAF, hệ thống cần được khởi động và cấu hình để đảm bảo mô hình Reverse Proxy hoạt động ổn định và lỗ hổng tồn tại nguyên vẹn.

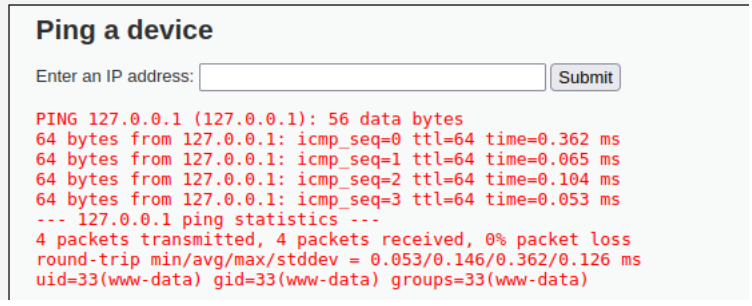


Hình 1 Kiểm tra hoạt động của DVWA và module ModSecurity

1.4.2 Bước 1 – Khai thác Command Injection khi chưa qua WAF (Off)

Trong giai đoạn này, mô đun kiểm tra của ModSecurity được vô hiệu hóa hoàn toàn nhằm mô phỏng một ứng dụng web truyền thống không có WAF bảo vệ.

Thực hiện: Kẻ tấn công truy cập vào chức năng thực thi lệnh của DVWA, nhập payload kết hợp toán tử nối lệnh hệ thống: 127.0.0.1 && id.



Hình 2 Khai thác Command Injection

Kết quả trên hình cho thấy khi nhập 127.0.0.1 && id, ứng dụng đã thực hiện cả hai lệnh: phần ping xác nhận máy chủ tự kết nối với chính nó thành công, và phần id in ra thông tin người dùng thực thi là www-data. Điều này chứng minh rõ ràng rằng ứng dụng web tồn

tại lỗ hổng command injection, cho phép kẻ tấn công chèn và chạy lệnh hệ thống dưới quyền tài khoản web server, gây ra rủi ro bảo mật nghiêm trọng cần được khắc phục ngay.

1.4.3 Bước 2 – Vận hành ModSecurity ở chế độ DetectionOnly với Custom Rule

Nhằm kiểm chứng khả năng giám sát thầm lặng, ModSecurity được chuyển sang chế độ DetectionOnly kết hợp với một luật tùy chỉnh chuyên biệt nhận diện ký tự đặc biệt của command injection.

Cấu hình áp dụng trên tệp trung tâm:

```
SecRuleEngine DetectionOnly


SecRequestBodyAccess On
SecResponseBodyAccess Off

SecRule ARGS "@rx (;|\||&|`|\$\()" \
    "id:10001,\
    phase:2,\
    deny,\
    status:403,\
    msg:'Command Injection Detected',\
    logdata:'Matched Data: %{TX.0}',\
    tag:'WEB_ATTACK/COMMAND_INJECTION'"
```

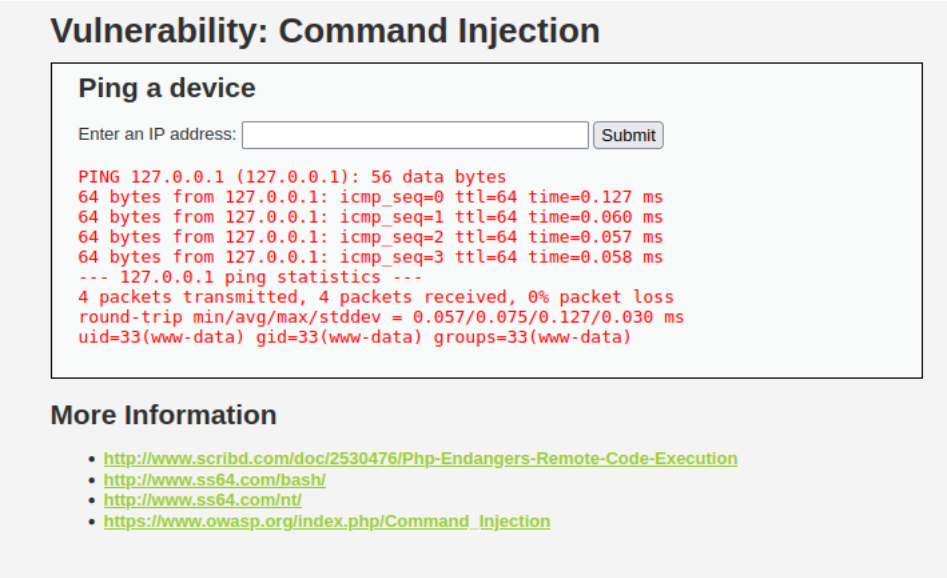
Hình 3 Cấu hình Custom Rule và DetectionOnly

Luật được thiết lập nhằm phát hiện và ngăn chặn hành vi command injection thông qua các tham số trong request. Cấu hình SecRuleEngine DetectionOnly cho phép hệ thống chạy ở chế độ chỉ phát hiện, tức là ghi nhận sự kiện nhưng chưa thực hiện chặn. Tiếp đó, SecRequestBodyAccess On bật khả năng phân tích nội dung body của request, trong khi SecResponseBodyAccess Off tắt việc phân tích phản hồi từ server.

Luật chính sử dụng cú pháp SecRule ARGS "@rx (;|\||&||$|())"để kiểm tra toàn bộ tham số request, tìm kiếm các ký tự đặc trưng thường xuất hiện trong tấn công command injection như dấu chấm phẩy, dấu gạch đứng, dấu “&” hoặc ký tự “$”. Khi phát hiện, hệ thống sẽ gán mã định danh id:10001, thực thi ở giai đoạn phân tích request (phase:2), từ chối truy cập với mã lỗi HTTP 403, đồng thời ghi log với thông điệp “Command Injection Detected” và dữ liệu khớp. Luật cũng gắn nhãnWEB_ATTACK/COMMAND_INJECTION` để phân loại sự kiện.

Như vậy, luật này đóng vai trò quan trọng trong việc giám sát và cảnh báo các yêu cầu có dấu hiệu tấn công command injection, giúp quản trị viên phát hiện sớm nguy cơ và có biện pháp xử lý phù hợp.

Thực hiện: Tiến hành gửi lại payload 127.0.0.1 && id trong khi mở terminal theo dõi tệp log hệ thống.



Hình 4 Payload vẫn thực thi thành công

Trên giao diện trình duyệt, lệnh vẫn chạy thành công và trả về kết quả định danh do WAF không can thiệp vào việc chặn request. Tuy nhiên, tại tệp log hệ thống, ModSecurity đã ghi nhận chính xác sự kiện vi phạm của Custom Rule ID 10001 kèm theo dữ liệu khớp mẫu.

```
---gqStcTgt---H--
ModSecurity: Warning. Matched "Operator `Rx' with parameter `(;|\||&|`|\$\()' against variable `ARGS:ip' (Value: `127.0.0.1 && id' )
[file "/etc/nginx/modsec_clean.conf"] [line "8"] [id "10001"] [rev ""] [msg "Command Injection Detected"] [data "Matched Data: "] [se
verity "0"] [ver ""] [maturity "0"] [accuracy "0"] [tag "WEB_ATTACK/COMMAND_INJECTION"] [hostname "127.0.0.1"] [uri "/vulnerabilities
/exec/"] [unique_id "179060457571.861511"] [ref "o10,1o10,1v658,15"]
```

Log ModSecurity này cho thấy hệ thống đã phát hiện một yêu cầu có dấu hiệu command injection. Cụ thể, tham số ARGS:ip chứa giá trị nhập vào là 127.0.0.1 && id, tức người dùng đã cố gắng chèn thêm lệnh hệ thống sau địa chỉ IP. Rule ID 10001 trong file cấu hình /etc/nginx/modsec_clean.conf đã khớp với chuỗi này, kích hoạt cảnh báo với thông điệp "Command Injection Detected" và gắn nhãn WEB_ATTACK/COMMAND_INJECTION. Log cũng ghi lại đầy đủ ngữ cảnh như URI /vulnerabilities/exec/, hostname, cùng với unique ID để phục vụ việc truy vết.

Kết luận rút ra là luật bảo mật đã hoạt động đúng, nhận diện chính xác hành vi bất thường và ghi log chi tiết, chứng minh rằng hệ thống WAF có khả năng phát hiện và cảnh báo kịp thời các nỗ lực tấn công command injection, giúp quản trị viên có cơ sở để xử lý và tăng cường bảo vệ ứng dụng web.

1.4.4 Bước 3 – Chuyển sang Blocking Mode (SecRuleEngine On) với Custom Rule

Sau khi xác thực độ chính xác của luật qua giai đoạn giám sát, hệ thống được chuyển sang trạng thái phòng thủ chủ động thực sự.

Cấu hình áp dụng: Thay đổi trạng thái engine trong tệp cấu hình trung tâm thành On:

```
SecRuleEngine On


SecRequestBodyAccess On
SecResponseBodyAccess Off

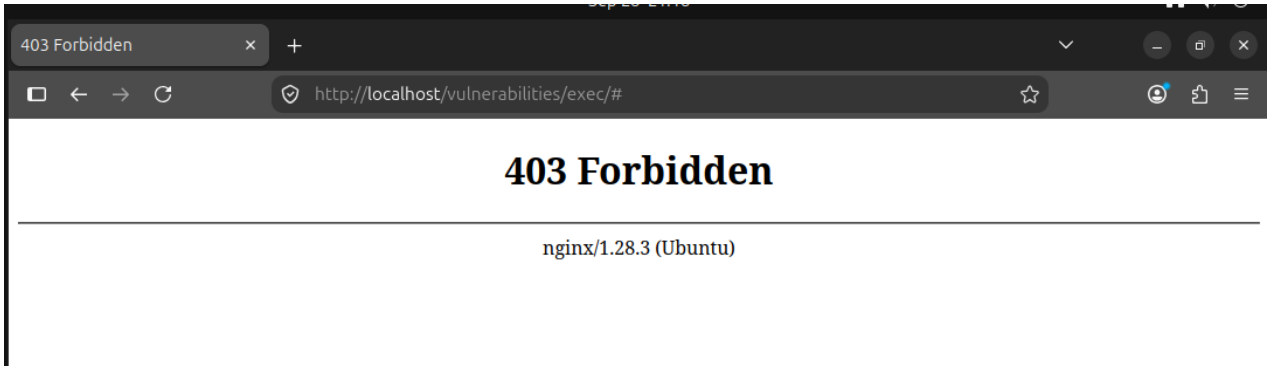
SecRule ARGS "@rx (;|\||&|`|\$\()" \
    "id:10001,\
    phase:2,\
    deny,\
    status:403,\
    msg:'Command Injection Detected',\
    logdata:'Matched Data: %{TX.0}',\
    tag:'WEB_ATTACK/COMMAND_INJECTION'"
```

Hình 5 Kích hoạt Blocking Mode

Thực hiện: Khởi động lại Nginx và thực hiện gửi lại payload tấn công trên DVWA.

```
nguyenhai@nguyenhai-VMware-Virtual-Platform:~$ sudo nginx -t && sudo systemctl restart nginx
2026/09/28 21:17:44 [notice] 26169#26169: ModSecurity-nginx v1.0.3 (rules loaded inline/local/remote: 0/1/0)
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

Hình 6 Khởi động lại Nginx



Hình 7 Chặn đứng request và trả về HTTP 403

Ngay lập tức, request bị chặn đứng tại cổng WAF trước khi kịp chạm tới ứng dụng DVWA, trình duyệt hiển thị mã lỗi HTTP 403 Forbidden. Lệnh hệ thống hoàn toàn không thể thực thi.

```
2026/09/28 21:18:48 [error] 26186#26186: *1 [client 127.0.0.1] ModSecurity: Access denied with code 403 (phase 2). Matched
"Operator `Rx' with parameter `(;|\||&|`|\$\()' against variable `ARGS:ip' (Value: `127.0.0.1 && id' ) [file "/etc/nginx/mo
dsec_clean.conf"] [line "8"] [id "10001"] [rev ""] [msg "Command Injection Detected"] [data "Matched Data: "] [severity "0"
] [ver ""] [maturity "0"] [accuracy "0"] [tag "WEB_ATTACK/COMMAND_INJECTION"] [hostname "127.0.0.1"] [uri "/vulnerabilities
/exec/"] [unique_id "179060512863.869717"] [ref "o10,1o10,1v658,15"], client: 127.0.0.1, server: _, request: "POST /vulnera
bilities/exec/ HTTP/1.1", host: "localhost", referrer: "http://localhost/vulnerabilities/exec/"
```

Hình 8 Log ghi nhận chặn

Log ModSecurity này cho thấy hệ thống đã từ chối truy cập với mã lỗi 403 do phát hiện một yêu cầu có dấu hiệu command injection. Cụ thể, tham số ARGS:ip chứa giá trị 127.0.0.1 && id, tức là người dùng đã cố tình chèn thêm lệnh hệ thống vào dữ liệu đầu vào. Rule ID 10001 trong file cấu hình /etc/nginx/modsec_clean.conf đã khớp với chuỗi này, kích hoạt cơ chế chặn ở phase 2 và ghi log với thông điệp "Command Injection Detected", đồng thời gắn nhãn WEB_ATTACK/COMMAND_INJECTION. Log cũng lưu lại đầy đủ thông tin ngữ cảnh như thời gian, client IP, URI /vulnerabilities/exec/, cùng unique ID để phục vụ việc truy vết.

Kết luận rút ra là hệ thống WAF không chỉ phát hiện mà còn chặn thành công nỗ lực tấn công command injection, đảm bảo an toàn cho ứng dụng web và cung cấp dữ liệu chi tiết để quản trị viên có thể phân tích, theo dõi và xử lý sự cố một cách hiệu quả.

1.4.5 Bước 4 – Kiểm thử độc lập năng lực phòng thủ của OWASP Core Rule Set

Nhằm kiểm chứng khả năng bảo vệ tổng quát của tập luật tiêu chuẩn quốc tế, luật tùy chỉnh được tạm thời vô hiệu hóa để đánh giá hiệu năng độc lập của OWASP CRS.

```
Include /etc/nginx/crs/crs-setup.conf
Include /etc/nginx/crs/rules/*.conf
```

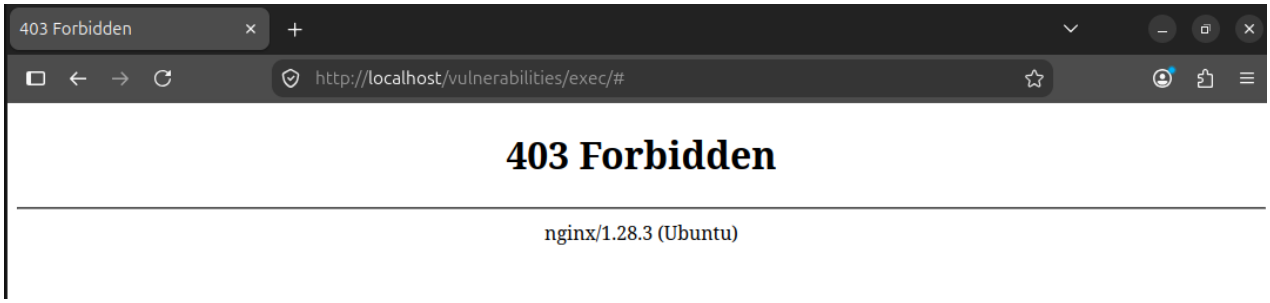
Hình 9 Kích hoạt OWASP CRS

Trạng thái tệp cấu hình chuyển sang sử dụng bộ quy tắc chuẩn quốc tế dưới chế độ SecRuleEngine On.

```
nguyenhai@nguyenhai-VMware-Virtual-Platform:~$ sudo nginx -t && sudo systemctl restart nginx
2026/09/28 21:25:14 [notice] 26364#26364: ModSecurity-nginx v1.0.3 (rules loaded inline/local/remote: 0/842/0)
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

Hình 10 Khởi động lại Nginx

Thực hiện: Reload Nginx và thực hiện lại các biến thể tấn công Command Injection.



Hình 11 OWASP CRS độc lập chặn request

Mặc dù không còn sự hỗ trợ từ luật tùy chỉnh, request tấn công vẫn bị ModSecurity ngăn chặn tuyệt đối với mã HTTP 403 Forbidden.

```
ModSecurity: Warning. Matched "Operator `Rx' with parameter `\$(?:\((?:[^\)]+|\([^\)]+\))\)|\{[^\}]+\}|\[[^\]]*\]
|[<>]\([^\)]+\)|/[0-9A-Z_a-z]*\[[^\]]+\]' against variable `ARGS:ip' (Value: `127.0.0.1 $(id)' ) [file "/etc/nginx
/crs/rules/REQUEST-932-APPLICATION-ATTACK-RCE.conf"] [line "297"] [id "932130"] [rev ""] [msg "Remote Command Exec
ution: Unix Shell Expression Found"] [data "Matched Data: $(id) found within ARGS:ip: 127.0.0.1 $(id)"] [severity
"2"] [ver "OWASP_CRS/4.30.0-dev"] [maturity "0"] [accuracy "0"] [tag "application-multi"] [tag "language-shell"] [
tag "platform-unix"] [tag "attack-rce"] [tag "paranoia-level/1"] [tag "OWASP_CRS"] [tag "OWASP_CRS/ATTACK-RCE"] [t
ag "capec/1000/152/248/88"] [hostname "127.0.0.1"] [uri "/vulnerabilities/exec/"] [unique_id "179060627220.260382"
] [ref "o10,5v658,15t:cmdLine"]
ModSecurity: Access denied with code 403 (phase 2). Matched "Operator `Ge' with parameter `5' against variable `TX
:BLOCKING_INBOUND_ANOMALY_SCORE' (Value: `5' ) [file "/etc/nginx/crs/rules/REQUEST-949-BLOCKING-EVALUATION.conf"]
[line "222"] [id "949110"] [rev ""] [msg "Inbound Anomaly Score Exceeded (Total Score: 5)"] [data ""] [severity "0
"] [ver "OWASP_CRS/4.30.0-dev"] [maturity "0"] [accuracy "0"] [tag "anomaly-evaluation"] [tag "OWASP_CRS"] [hostna
me "127.0.0.1"] [uri "/vulnerabilities/exec/"] [unique_id "179060627220.260382"] [ref ""]
```

Hình 12 Phân tích log OWASP CRS

Log ModSecurity này cho thấy hệ thống đã phát hiện và chặn một nỗ lực remote command execution thông qua biểu thức shell Unix. Trong bản ghi đầu tiên, tham số ARGS:ip chứa giá trị 127.0.0.1 $(id), tức là người dùng đã chèn thêm lệnh id bằng cú pháp

$() để thực thi trên hệ thống. Rule ID 932130 trong file REQUEST-932-APPLICATION-ATTACK-RCE.conf đã khớp với chuỗi này và đưa ra cảnh báo "Remote Command Execution: Unix Shell Expression Found", đồng thời ghi rõ dữ liệu khớp là $(id). Đây là minh chứng cho việc ModSecurity nhận diện chính xác hành vi command injection dưới dạng RCE. Bản ghi tiếp theo cho thấy rule ID 949110 trong file REQUEST-949-BLOCKING-EVALUATION.conf đã kích hoạt vì tổng điểm bất thường đạt ngưỡng 5, dẫn đến việc từ chối truy cập với mã lỗi 403.

Kết luận rút ra là hệ thống WAF đã hoạt động đúng, phát hiện và chặn thành công một yêu cầu có dấu hiệu command injection thông qua biểu thức shell, đồng thời áp dụng cơ chế đánh giá điểm bất thường để đảm bảo an toàn cho ứng dụng web. Đây là bằng chứng rõ ràng cho hiệu quả của các rule OWASP CRS trong việc bảo vệ chống lại tấn công command injection.

1.5 Kết luận

Kết luận rút ra từ toàn bộ quá trình thực nghiệm là hệ thống WAF sử dụng Nginx kết hợp ModSecurity đã chứng minh năng lực phòng thủ hiệu quả trước lỗ hổng Command Injection trên ứng dụng DVWA. Ở giai đoạn không có WAF, payload độc hại được thực thi trực tiếp, khẳng định ứng dụng tồn tại lỗ hổng nghiêm trọng. Khi chuyển sang chế độ DetectionOnly với luật tùy chỉnh, ModSecurity đã ghi nhận chính xác sự kiện tấn công trong log, cho thấy khả năng giám sát và cảnh báo. Tiếp đó, ở chế độ Blocking với luật tùy chỉnh, hệ thống đã chủ động chặn đứng request, trả về mã lỗi 403 và ngăn không cho lệnh hệ thống thực thi. Cuối cùng, khi vô hiệu hóa luật tùy chỉnh và chỉ sử dụng OWASP CRS, WAF vẫn phát hiện và ngăn chặn thành công các biến thể command injection thông qua cơ chế chấm điểm bất thường.

Như vậy, báo cáo đã chứng minh rằng việc triển khai ModSecurity cùng với OWASP CRS không chỉ giúp nhận diện mà còn ngăn chặn hiệu quả các nỗ lực khai thác command injection, đảm bảo an toàn cho ứng dụng web và cung cấp log chi tiết phục vụ phân tích sự cố. Đây là minh chứng rõ ràng cho vai trò quan trọng của WAF như một lớp phòng thủ tuyến đầu trong bảo mật hệ thống.